Lab 14: Web Frontend Development – AI-Assisted HTML/CSS/JS Generation

Student Details

Student Name: K.SAI VARDHAN REDDY

Admission No: 2503A51266

Course: AI Assisted Coding (25CS002PC304)

Task 1: Responsive Homepage Layout for a Startup Website

AI Prompt Used: "Generate a responsive homepage layout for a startup website using HTML5 semantic tags (header, nav, main, footer). Use CSS media queries to ensure the layout adjusts smoothly between desktop and mobile screen sizes."

HTML/CSS Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Startup Homepage</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 0; padding: 0; }
        header { background-color: #333; color: white; padding: 1rem; text-align: center; }
        nav ul { list-style: none; padding: 0; margin: 0; display: flex; justify-content: center; }
        nav ul li { margin: 0 15px; }
        nav ul li a { color: white; text-decoration: none; }
        main { padding: 2rem; text-align: center; min-height: 60vh; }
        footer { background-color: #333; color: white; text-align: center; padding: 1rem; position: relative; bottom: 0; width: 100%; }

        /* Responsive Design */
        @media (max-width: 600px) {
            nav ul { flex-direction: column; }
            nav ul li { margin: 10px 0; }
        }
    </style>
</head>
<body>
    <header>
        <h1>Tech Startup</h1>
        <nav>
            <ul>
                <li><a href="#">Home</a></li>
                <li><a href="#">About</a></li>
                <li><a href="#">Contact</a></li>
            </ul>
        </nav>
    </header>
    <main>
        <h2>Welcome to the Future</h2>
        <p>We build innovative AI solutions for modern problems.</p>
    </main>
    <footer>
        <p>&copy; 2026 Tech Startup. All rights reserved.</p>
    </footer>
</body>
</html>
```

Code Explanation:

The HTML uses semantic tags (header, main, footer) to clearly separate content areas. CSS Flexbox is used in the navigation bar to distribute links horizontally. A CSS media query (@media (max-width: 600px)) detects mobile screens and automatically switches the flex direction from row to column, ensuring buttons stack vertically and remain touch-friendly on smaller devices.

Task 2: Interactive Call-to-Action Button

AI Prompt Used: "Create an HTML call-to-action button. Write well-commented JavaScript that displays a visible message directly on the page (without reloading) when the button is clicked."

HTML/JS Code:

```html
<!DOCTYPE html>
<html lang="en">
<body>
    <button id="ctaButton" style="padding: 10px 20px; font-size: 16px;">Click Me to Start</button>
    <p id="responseMessage" style="color: green; font-weight: bold;"></p>

    <script>
        // 1. Select the button and the message display area from the DOM
        const button = document.getElementById('ctaButton');
        const messageArea = document.getElementById('responseMessage');

        // 2. Attach an event listener that triggers when the button is clicked
        button.addEventListener('click', function() {
            // 3. Update the text content of the message area, proving interactivity without reload
            messageArea.textContent = 'Action initiated! Welcome aboard.';
        });
    </script>
</body>
</html>
```

Code Explanation:

An HTML button is paired with an empty paragraph tag (p) to act as a placeholder. The JavaScript logic uses document.getElementById to grab these elements. By attaching an 'addEventListener' specifically for the 'click' event, the script updates the paragraph's textContent property directly in the DOM. This provides immediate user feedback without initiating a page refresh.

Task 3: Contact Form with Client-Side Validation

AI Prompt Used: "Design an HTML contact form (Name, Email, Message). Use JavaScript to add client-side validation that prevents empty submissions and checks for valid email formatting. Display inline error messages for invalid input and a success message for valid input."

HTML/JS Code:

```html
<!DOCTYPE html>
<html lang="en">
<body>
    <form id="contactForm">
        <div>
            <label>Name:</label>
            <input type="text" id="nameInput">
        </div>
        <div>
            <label>Email:</label>
            <input type="text" id="emailInput">
        </div>
        <div>
            <label>Message:</label>
            <textarea id="msgInput"></textarea>
        </div>
        <button type="submit">Submit</button>
        <p id="feedback" style="color: red;"></p>
    </form>

    <script>
        document.getElementById('contactForm').addEventListener('submit', function(e) {
            // Prevent the default form submission (page reload)
            e.preventDefault();

            const name = document.getElementById('nameInput').value.trim();
            const email = document.getElementById('emailInput').value.trim();
            const msg = document.getElementById('msgInput').value.trim();
            const feedback = document.getElementById('feedback');

            // Basic Regex for Email Validation
            const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

            if (!name || !email || !msg) {
                feedback.style.color = 'red';
                feedback.textContent = 'Error: All fields are required.';
            } else if (!emailRegex.test(email)) {
                feedback.style.color = 'red';
                feedback.textContent = 'Error: Please enter a valid email address.';
            } else {
                feedback.style.color = 'green';
                feedback.textContent = 'Success! Your message has been sent.';
                this.reset(); // Clear form fields
            }
        });
    </script>
</body>
</html>
```

Code Explanation:

The script begins with e.preventDefault() to intercept the standard form submission process, ensuring validation occurs strictly on the client side. String method .trim() is used to prevent users from bypassing empty checks using spaces. A Regular Expression (regex) pattern validates the structural logic of the email address (text@text.domain). The feedback message styling and content dynamically update based on the if-else validation tree.

Task 4: Dynamic List Management for a Product Page

AI Prompt Used: "Create a simple HTML list. Write JavaScript that dynamically adds a new item to the list when an "Add" button is clicked, and removes the last item when a "Remove" button is clicked, updating the DOM dynamically."

HTML/JS Code:

```html
<!DOCTYPE html>
<html lang="en">
<body>
    <h3>Product List</h3>
    <ul id="productList">
        <li>Item 1</li>
    </ul>
    <button id="addItemBtn">Add Item</button>
    <button id="removeItemBtn">Remove Item</button>

    <script>
        const list = document.getElementById('productList');
        let counter = 2; // Keeps track of new item numbers

        document.getElementById('addItemBtn').addEventListener('click', () => {
            // Create a new HTML element dynamically
            const newItem = document.createElement('li');
            newItem.textContent = `Item ${counter}`;
            list.appendChild(newItem); // Append to the end of the list
            counter++;
        });

        document.getElementById('removeItemBtn').addEventListener('click', () => {
            // Check if list has items before trying to remove
            if (list.lastElementChild) {
                list.removeChild(list.lastElementChild); // Remove the last node
                counter--;
            }
        });
    </script>
</body>
</html>
```

Code Explanation:

This script manipulates the Document Object Model (DOM) structurally. The 'Add' button utilizes document.createElement('li') to generate a new node in memory, assigns text, and then pushes it into the live DOM using .appendChild(). The 'Remove' button accesses the .lastElementChild property of the list and deletes it via .removeChild(). Both operations happen instantaneously without network requests.

Task 5: Modal Popup for User Notifications

AI Prompt Used: "Generate the HTML structure, CSS styling with a semi-transparent background overlay, and JavaScript logic to create a modal popup. Allow the user to close the modal by clicking a close button or clicking the background overlay."

HTML/CSS/JS Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <style>
        /* Modal Overlay Background */
        .modal-overlay {
            display: none; /* Hidden by default */
            position: fixed; top: 0; left: 0; width: 100%; height: 100%;
            background: rgba(0, 0, 0, 0.6); /* Semi-transparent black */
            justify-content: center; align-items: center; z-index: 1000;
        }
        /* Actual Popup Box */
        .modal-content {
            background: white; padding: 20px; border-radius: 8px;
            text-align: center; width: 300px; position: relative;
        }
        .close-btn { margin-top: 15px; cursor: pointer; }
    </style>
</head>
<body>
    <button id="openModal">Open Modal</button>

    <!-- Modal Structure -->
    <div id="myModal" class="modal-overlay">
        <div class="modal-content">
            <h2>Notification</h2>
            <p>This is an important alert for users!</p>
            <button id="closeModal" class="close-btn">Close</button>
        </div>
    </div>

    <script>
        const modal = document.getElementById('myModal');
        const openBtn = document.getElementById('openModal');
        const closeBtn = document.getElementById('closeModal');

        // 1. Open the modal by switching CSS display property to Flex
        openBtn.addEventListener('click', () => {
            modal.style.display = 'flex';
        });

        // 2. Close modal via button
        closeBtn.addEventListener('click', () => {
            modal.style.display = 'none';
        });

        // 3. Close modal when clicking on the semi-transparent overlay
        window.addEventListener('click', (e) => {
            if (e.target === modal) {
                modal.style.display = 'none';
            }
        });
    </script>
</body>
</html>
```

Code Explanation:

The modal's visual hierarchy is established using CSS absolute/fixed positioning and a high z-index to overlay standard page content. The background utilizes 'rgba' for partial transparency. JavaScript controls the visibility toggle by flipping the display style between 'none' and 'flex'. Crucially, the window click listener verifies if the exact element clicked (e.target) was the overlay background itself, allowing intuitive closure without hitting the exact button.